



MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software and Automation Engineering

Zavtoni Ion

Group: FAF-252

Report

Laboratory Work No.5

The variant No.26

of the "Data Structures and Algorithms" course

Checked:

Burlacu Natalia, PhD, associate professor

Department of Software and Automation Engineering,

Facultatea FCIM, UTM

Chisinau – 2026

1. The purpose of the laboratory work

The objective of this laboratory work is to develop practical skills in procedural C programming by utilizing dynamic memory allocation, pointer manipulation, and User-Defined Types (UDTs). The assignment aims to consolidate knowledge of file handling (reading and writing text files) and applying fundamental algorithms, such as sorting dynamically allocated structures, while ensuring efficient memory management without leaks.

2. Problem conditions

5.1 Solving the problems using FILE data type

Write a C program to perform the following operations:

- Create an input text file (`input.txt`) and store a string read from the keyboard.
- Read the string back from the file.
- Write a function to interchange two sentences of the string (sentence 1 and sentence at a given *step*). The step of swaps should be read from the keyboard.
- Display the string before and after changes, and write the results to `output.txt`.

5.2 Solving the problems using FILE data type & UDT

Develop a Registry of Citizens containing: name, surname, date of birth, gender, home address, work address (city, street, post office code).

- Create `experiment.txt` and store entered UDT data.
- Calculate age in years, months, and days based on the current date, and assign a category and amount paid.
- Enter the calculations into `output.txt`, sorted in descending order according to their category.
- Merge `output.txt` & `experiment.txt` into `result.txt`.

3. The program code

This section presents the complete C source code developed for the laboratory assignment. The implementation is divided into two distinct procedural programs: Part 1 handles dynamic sentence manipulation, and Part 2 focuses on a Citizen Registry system utilizing UDTs and dynamic pointer arrays.

————— Part 1: Dynamic Sentence Manipulation —————

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_BUFFER 2048

/* Function parseSentences
   Dynamically parses a block of text into individual sentences.
   Returns dynamically allocated array of string pointers (char**).
*/
char** parseSentences(char* text, int* count) {
    int capacity = 10;

    // Allocate initial memory for an array of 10 pointers
    char** sentences = (char**)malloc(capacity * sizeof(char*));
    *count = 0;

    // Use two pointers to track the beginning and current position of a sentence
    char* start = text;
    char* current = text;

    while (*current != '\0') {
        // Skip any leading spaces at the start of a new sentence
        if (start == current && *current == ' ') {
            start++;
            current++;
            continue;
        }

        // Check for end of sentence punctuation
        if (*current == '.' || *current == '!' || *current == '?') {
            // Calculate exact length using pointer arithmetic
            int len = current - start + 1;
```

```

// Dynamically allocate memory for this specific sentence
sentences[*count] = (char*)malloc((len + 1) * sizeof(char));

// Copy the extracted sentence into the allocated memory
strncpy(sentences[*count], start, len);
sentences[*count][len] = '\0';
(*count)++;

// If we exceed our pointer array capacity, double the size using
realloc
if (*count >= capacity) {
    capacity *= 2;
    sentences = (char**)realloc(sentences, capacity * sizeof(char*));
}

// Move pointers forward to start scanning the next sentence
current++;
start = current;
} else {
    current++;
}
}

// special case that matches any leftover text at the end
if (current > start) {
    int len = current - start;
    sentences[*count] = (char*)malloc((len + 1) * sizeof(char));
    strncpy(sentences[*count], start, len);
    sentences[*count][len] = '\0';
    (*count)++;
}

return sentences;
}

/* Function interchangeSentences
   Swaps two sentences based on the user's step index.
   This swaps memory addresses instead of copying strings for O(1) efficiency.
*/
void interchangeSentences(char** sentences, int count, int step) {
    // Validate the user input to prevent out-of-bounds array access

```

```

    if (step < 1 || step > count) {
        printf("Error: Step %d is out of range.\n", step);
        return;
    }

    // Swap the pointers directly using a temporary pointer
    char* temp = sentences[0];
    sentences[0] = sentences[step - 1];
    sentences[step - 1] = temp;
}

/* Function joinSentences
   Reassembles the dynamically allocated array of sentences back into a single
   string.
*/
void joinSentences(char** sentences, int count, char* result) {
    result[0] = '\0';
    for (int i = 0; i < count; i++) {
        if (i > 0) {
            strcat(result, " "); // Add a space between sentences
        }
        strcat(result, sentences[i]);
    }
}

int main(void) {
    char inputBuffer[MAX_BUFFER];
    char beforeStr[MAX_BUFFER];
    char afterStr[MAX_BUFFER];
    int step;

    // --- Step 1: Read input from keyboard ---
    printf("Introduce the string:\n> ");
    fgets(inputBuffer, MAX_BUFFER, stdin);
    inputBuffer[strcspn(inputBuffer, "\n")] = '\0';

    // --- Step 2: Write standard input to input.txt ---
    FILE* fileOut = fopen("input.txt", "w");
    if (!fileOut) return 1;
    fprintf(fileOut, "%s", inputBuffer);
    fclose(fileOut);
}

```

```

// --- Step 3: Read data back from input.txt ---
FILE* fileIn = fopen("input.txt", "r");
if (!fileIn) return 1;
fgets(inputBuffer, MAX_BUFFER, fileIn);
fclose(fileIn);

printf("\nInput string is:\n%s\n", inputBuffer);

// --- Step 4: Get swap index from user ---
printf("\nIntroduce your step of swaps: ");
scanf("%d", &step);

// --- Step 5: Process data using dynamic memory ---
int count = 0;
char** sentences = parseSentences(inputBuffer, &count);

// --- Step 6: Build and display the original text ---
joinSentences(sentences, count, beforeStr);
printf("\n--- Before changes ---\n%s\n", beforeStr);

// --- Step 7: Perform pointer swap and display updated text ---
interchangeSentences(sentences, count, step);
joinSentences(sentences, count, afterStr);
printf("\n--- After changes ---\n%s\n", afterStr);

// --- Step 8: Save final results to output.txt ---
FILE* finalOut = fopen("output.txt", "w");
if (!finalOut) return 1;
fprintf(finalOut, "Before changes:\n%s\n\nAfter changes:\n%s\n", beforeStr,
        afterStr);
fclose(finalOut);
printf("\nResults saved to output.txt\n");

// --- Step 9: Clean up dynamically allocated memory ---
for (int i = 0; i < count; i++) {
    free(sentences[i]);
}
free(sentences);

return 0;
}

```

———— Part 2: Citizen Registry System (UDT) ————

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// --- User-Defined Types ---
// UDTs allow to group related variables of different data types into a single
// logical unit.

typedef struct {
    int day;
    int month;
    int year;
} Date;

typedef struct {
    char city[50];
    char street[50];
    char postCode[20];
} Address;

// The main Citizen UDT bundles primitive types with other UDTs (Date, Address)
typedef struct {
    char name[50];
    char surname[50];
    Date dob;
    char gender;
    Address home;
    Address work;

    // Additional fields required for calculation and sorting operations
    int ageYears;
    int ageMonths;
    int ageDays;
    int category;
    float amountPaid;
} Citizen;

/* --- Function Prototypes --- */
void clearBuffer();
Citizen** createCitizens(int count);
void calculateAgeAndCategory(Citizen* c, Date current);
```

```

void displayCitizen(Citizen* c);
void sortCitizensByCategory(Citizen** registry, int count);
void mergeFiles(const char* file1, const char* file2, const char* fileOut);

int main() {
    int count;
    Date currentDate;

    printf("---- Citizen Registry System ----\n");
    printf("Enter the number of citizens to register: ");
    scanf("%d", &count);
    clearBuffer();

    // --- Dynamic Memory Allocation for Pointer Array ---
    // allocate an array of POINTERS (Citizen**) to Citizen structs.
    Citizen** registry = createCitizens(count);

    // --- Open experiment.txt using FILE type ---
    FILE* expFile = fopen("experiment.txt", "w");
    if (!expFile) return 1;

    for (int i = 0; i < count; i++) {
        // Allocate specific memory space for each individual citizen struct
        registry[i] = (Citizen*)malloc(sizeof(Citizen));

        printf("\n--- Entering data for Citizen %d ---\n", i + 1);

        printf("Name: ");
        fgets(registry[i]->name, 50, stdin);
        registry[i]->name[strcspn(registry[i]->name, "\n")] = 0;

        printf("Surname: ");
        fgets(registry[i]->surname, 50, stdin);
        registry[i]->surname[strcspn(registry[i]->surname, "\n")] = 0;

        printf("Gender (m/f): ");
        scanf("%c", &registry[i]->gender);

        printf("Date of Birth (DD MM YYYY): ");
        scanf("%d %d %d", &registry[i]->dob.day, &registry[i]->dob.month, &
            registry[i]->dob.year);
        clearBuffer();
    }
}

```



```

// --- FULL HOME ADDRESS ---
printf("\n[ Home Address ]\n");
printf("City: ");
fgets(registry[i]->home.city, 50, stdin);
registry[i]->home.city[strcspn(registry[i]->home.city, "\n")] = 0;

printf("Street: ");
fgets(registry[i]->home.street, 50, stdin);
registry[i]->home.street[strcspn(registry[i]->home.street, "\n")] = 0;

printf("Postal Code (e.g., MD-4580): ");
fgets(registry[i]->home.postCode, 20, stdin);
registry[i]->home.postCode[strcspn(registry[i]->home.postCode, "\n")] = 0;

// --- FULL WORK ADDRESS ---
printf("\n[ Work Address ]\n");
printf("City: ");
fgets(registry[i]->work.city, 50, stdin);
registry[i]->work.city[strcspn(registry[i]->work.city, "\n")] = 0;

printf("Street: ");
fgets(registry[i]->work.street, 50, stdin);
registry[i]->work.street[strcspn(registry[i]->work.street, "\n")] = 0;

printf("Postal Code: ");
fgets(registry[i]->work.postCode, 20, stdin);
registry[i]->work.postCode[strcspn(registry[i]->work.postCode, "\n")] = 0;

// Write the structured raw data directly to the file stream
fprintf(expFile, "%s %s | Gender: %c | DOB: %02d/%02d/%04d\n",
        registry[i]->name, registry[i]->surname, registry[i]->gender,
        registry[i]->dob.day, registry[i]->dob.month, registry[i]->dob.year)
        ;
fprintf(expFile, " Home: %s, str. %s, %s\n",
        registry[i]->home.city, registry[i]->home.street, registry[i]->home.
        postCode);
fprintf(expFile, " Work: %s, str. %s, %s\n\n",
        registry[i]->work.city, registry[i]->work.street, registry[i]->work.
        postCode);
}
fclose(expFile);

```

```

// --- Display entered data ---
printf("\n=== Data Saved to experiment.txt ===\n");
for (int i = 0; i < count; i++) {
    displayCitizen(registry[i]);
}

// --- Data Processing (Age, Category, Amount Paid) ---
printf("\nEnter the CURRENT DATE (DD MM YYYY) to calculate ages and amounts:
");
scanf("%d %d %d", &currentDate.day, &currentDate.month, &currentDate.year);

for (int i = 0; i < count; i++) {
    calculateAgeAndCategory(registry[i], currentDate);
}

// --- Sort descending by category using pointers ---
sortCitizensByCategory(registry, count);

// --- Write processed calculations to output.txt ---
FILE* outFile = fopen("output.txt", "w");
if (!outFile) return 1;

fprintf(outFile, "--- Processed Citizens (Sorted by Category Descending) ---\n");
for (int i = 0; i < count; i++) {
    fprintf(outFile, "Cat: %d | %s %s | Age: %d Y, %d M, %d D | Paid: %.2f |
        Home: %s | Work: %s\n",
        registry[i]->category, registry[i]->name, registry[i]->surname,
        registry[i]->ageYears, registry[i]->ageMonths, registry[i]->ageDays,
        registry[i]->amountPaid, registry[i]->home.city, registry[i]->work.
        city);
}
fclose(outFile);
printf("\nProcessed calculations saved to output.txt.\n");

// --- Merge files ---
mergeFiles("experiment.txt", "output.txt", "result.txt");
printf("Merged experiment.txt and output.txt into result.txt.\n");

// --- Memory Cleanup ---
for (int i = 0; i < count; i++) free(registry[i]);

```

```

    free(registry);

    return 0;
}

/* --- Function Implementations --- */

void clearBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF) { }
}

Citizen** createCitizens(int count) {
    return (Citizen**)malloc(count * sizeof(Citizen*));
}

void displayCitizen(Citizen* c) {
    printf("Citizen: %s %s, Gender: %c, DOB: %02d/%02d/%04d\n",
           c->name, c->surname, c->gender, c->dob.day, c->dob.month, c->dob.year);
    printf(" Home: %s, %s, %s\n", c->home.city, c->home.street, c->home.postCode)
        ;
    printf(" Work: %s, %s, %s\n", c->work.city, c->work.street, c->work.postCode)
        ;
}

void calculateAgeAndCategory(Citizen* c, Date current) {
    c->ageDays = current.day - c->dob.day;
    c->ageMonths = current.month - c->dob.month;
    c->ageYears = current.year - c->dob.year;

    if (c->ageDays < 0) {
        c->ageMonths--;
        c->ageDays += 30;
    }
    if (c->ageMonths < 0) {
        c->ageYears--;
        c->ageMonths += 12;
    }

    if (c->ageYears < 18) {
        c->category = 1; c->amountPaid = 500.0;
    } else if (c->ageYears >= 18 && c->ageYears < 65) {

```

```

        c->category = 2; c->amountPaid = 1500.0;
    } else {
        c->category = 3; c->amountPaid = 2500.0;
    }
}

/* * Function: sortCitizensByCategory
 * Sorts the array of pointers using Bubble Sort (descending order).
 * Provides an O(1) swap complexity by swapping memory addresses.
 */
void sortCitizensByCategory(Citizen** registry, int count) {
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (registry[j]->category < registry[j + 1]->category) {
                Citizen* temp = registry[j];
                registry[j] = registry[j + 1];
                registry[j + 1] = temp;
            }
        }
    }
}

void mergeFiles(const char* file1, const char* file2, const char* fileOut) {
    FILE *f1 = fopen(file1, "r");
    FILE *f2 = fopen(file2, "r");
    FILE *fOut = fopen(fileOut, "w");
    char buffer[256];

    if (!f1 || !f2 || !fOut) return;

    fprintf(fOut, "=== DATA FROM EXPERIMENT.TXT ===\n");
    while (fgets(buffer, sizeof(buffer), f1)) fputs(buffer, fOut);

    fprintf(fOut, "\n=== DATA FROM OUTPUT.TXT ===\n");
    while (fgets(buffer, sizeof(buffer), f2)) fputs(buffer, fOut);

    fclose(f1); fclose(f2); fclose(fOut);
}

```

4. Program execution screenshots

```
PS C:\Users\Admin\Desktop\Coding\SDA\Lab_5> cd "c:\Users\Admin\Desktop\Coding\SDA\Lab_5\" ; if ($?) { gcc Part1.c -o Part1 } ; if ($?) { .\Part1 }
Introduce the string:
> I am studying at TUM. I started my studies in 2022. I will study for 4 years. I will become a software engineer.

Input string is:
I am studying at TUM. I started my studies in 2022. I will study for 4 years. I will become a software engineer.

Introduce your step of swaps: 4

--- Before changes ---
I am studying at TUM. I started my studies in 2022. I will study for 4 years. I will become a software engineer.

--- After changes ---
I will become a software engineer. I started my studies in 2022. I will study for 4 years. I am studying at TUM.

Results saved to output.txt
PS C:\Users\Admin\Desktop\Coding\SDA\Lab_5> █
```

Figure 4.1 – Terminal execution trace demonstrating the dynamic parsing of the input buffer and the successful $O(1)$ interchange of sentence pointers.

```
output.txt
1 Before changes:
2 I am studying at TUM. I started my studies in 2022. I will study for 4 years. I will become a software engineer.
3
4 After changes:
5 I will become a software engineer. I started my studies in 2022. I will study for 4 years. I am studying at TUM.
6
```

Figure 4.2 – Verification of the output .txt file contents, illustrating successful data persistence and structural consistency of the interchanged records.

```

PS C:\Users\Admin\Desktop\Coding\SDA\Lab_5> cd "c:\Users\Admin\Desktop\Coding\SDA\Lab_5\" ; if ($?) { gcc Part2.c -o Part2 } ; if ($?) { .\Part2 }
--- Citizen Registry System ---
Enter the number of citizens to register: 3

--- Entering data for Citizen 1 ---
Name: Iustin
Surname: Rusu
Gender (m/f): m
Date of Birth (DD MM YYYY): 15 05 2010

[ Home Address ]
City: Chisinau
Street: Independentei
Postal Code (e.g., MD-4580): MD-2001

[ Work Address ]
City: Chisinau
Street: Renasterii
Postal Code: MD-2100

--- Entering data for Citizen 2 ---
Name: Maria
Surname: Popa
Gender (m/f): f
Date of Birth (DD MM YYYY): 20 08 1995

[ Home Address ]
City: Balti
Street: Mihai Eminescu
Postal Code (e.g., MD-4580): MD-3101

[ Work Address ]
City: Balti
Street: Ion Creanga
Postal Code: MD-3112

--- Entering data for Citizen 3 ---
Name: Alexandru
Surname: Bradu
Gender (m/f): m
Date of Birth (DD MM YYYY): 10 02 1955

[ Home Address ]
City: Dubasari
Street: Scolii
Postal Code (e.g., MD-4580): MD-4580

[ Work Address ]
City: Chisinau
Street: Gheorghe Casu
Postal Code: MD-2055

=== Data Saved to experiment.txt ===
Citizen: Iustin Rusu, Gender: m, DOB: 15/05/2010
  Home: Chisinau, Independentei, MD-2001
  Work: Chisinau, Renasterii, MD-2100
Citizen: Maria Popa, Gender: f, DOB: 20/08/1995
  Home: Balti, Mihai Eminescu, MD-3101
  Work: Balti, Ion Creanga, MD-3112
Citizen: Alexandru Bradu, Gender: m, DOB: 10/02/1955
  Home: Dubasari, Scolii, MD-4580
  Work: Chisinau, Gheorghe Casu, MD-2055

Enter the CURRENT DATE (DD MM YYYY) to calculate ages and amounts: 09 04 2026

Processed calculations saved to output.txt.
Merged experiment.txt and output.txt into result.txt.
PS C:\Users\Admin\Desktop\Coding\SDA\Lab_5>

```

Figure 4.3 – Terminal interface displaying the structured acquisition of citizen records and subsequent age evaluations.

```

≡ experiment.txt
1  Iustin Rusu | Gender: m | DOB: 15/05/2010
2  | Home: Chisinau, str. Independentei, MD-2001
3  | Work: Chisinau, str. Renasterii, MD-2100
4
5  Maria Popa | Gender: f | DOB: 20/08/1995
6  | Home: Balti, str. Mihai Eminescu, MD-3101
7  | Work: Balti, str. Ion Creanga, MD-3112
8
9  Alexandru Bradu | Gender: m | DOB: 10/02/1955
10 | Home: Dubasari, str. Scolii, MD-4580
11 | Work: Chisinau, str. Gheorghe Casu, MD-2055
12
13 |

```

Figure 4.4 – Contents of `experiment.txt`, verifying the structural integrity of the raw UDT data saved to the local disk.

```

≡ result.txt
1  === DATA FROM EXPERIMENT.TXT ===
2  Iustin Rusu | Gender: m | DOB: 15/05/2010
3  | Home: Chisinau, str. Independentei, MD-2001
4  | Work: Chisinau, str. Renasterii, MD-2100
5
6  Maria Popa | Gender: f | DOB: 20/08/1995
7  | Home: Balti, str. Mihai Eminescu, MD-3101
8  | Work: Balti, str. Ion Creanga, MD-3112
9
10 Alexandru Bradu | Gender: m | DOB: 10/02/1955
11 | Home: Dubasari, str. Scolii, MD-4580
12 | Work: Chisinau, str. Gheorghe Casu, MD-2055
13
14
15 === DATA FROM OUTPUT.TXT ===
16 --- Processed Citizens (Sorted by Category Descending) ---
17 Cat: 3 | Alexandru Bradu | Age: 71 Y, 1 M, 29 D | Paid: 2500.00 | Home: Dubasari | Work: Chisinau
18 Cat: 2 | Maria Popa | Age: 30 Y, 7 M, 19 D | Paid: 1500.00 | Home: Balti | Work: Balti
19 Cat: 1 | Iustin Rusu | Age: 15 Y, 10 M, 24 D | Paid: 500.00 | Home: Chisinau | Work: Chisinau
20

```

Figure 4.5 – Verification of the merged `result.txt` file, demonstrating the successful descending sort by age category.

5. Conclusions

This laboratory work was implemented in C to demonstrate advanced procedural programming techniques, focusing heavily on dynamic memory allocation, User-Defined Types (UDTs), and file stream manipulation (`FILE`). The code was organized in a procedural style using clearly separated helper functions, and memory management aspects were treated carefully. The pointer-based implementations rely on dynamic allocation where appropriate and demonstrate the safe cleanup of all allocated resources using `free()` to prevent memory leaks.

In the first part of the assignment, the program dynamically parses a continuous string into an array of sentence pointers (`char**`). By utilizing memory allocation functions (`malloc` and `realloc`), the implementation ensures that memory is allocated precisely according to the input length, avoiding wasteful static buffers. The sentence interchange logic was optimized to $O(1)$ complexity by swapping the pointer memory addresses directly, rather than executing expensive string copy operations. Furthermore, the data flow was securely managed by persisting the standard input to `input.txt` and writing the final swapped outcome to `output.txt`.

To satisfy the second part of the assignment, the complexity was increased by introducing nested UDTs (`struct`) to model a Citizen Registry. The program bundled primitive data types with complex structures like `Date` and `Address` (containing detailed fields for city, street, and postal codes). To manage the records efficiently, an array of pointers to `Citizen` structs was allocated dynamically. The program then processed mathematical logic to calculate exact ages from the current date, assigning corresponding age categories and dummy payments based on the results.

For the data organization and persistence requirements, a Bubble Sort algorithm was implemented to order the citizens descendingly by category. Similar to the first part, this sorting was highly optimized by swapping struct pointers instead of moving heavy structural data in memory. Finally, the program demonstrated advanced file handling by maintaining concurrent read and write streams to successfully merge `experiment.txt` and `output.txt` into a unified `result.txt` document.

Overall, this work provided strong practice with dynamic memory management, pointer arithmetic, nested data structures, and the application of procedural File I/O operations in both algorithmic and record-processing contexts.